

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

Project Name: Duit On Track

Version 1.0

Table Of Content

1. Introduction.....	3
1.1. Purpose.....	4
1.2. Background.....	5
1.3. Target Stakeholder.....	7
2. System Architecture & Design.....	8
2.1. High Level Architecture.....	8
2.1.1. Overview:.....	8
2.1.2 LLM as Service Layer.....	9
2.1.3 Dependency Diagram.....	9
2.2.2. Sequence Diagram.....	12
2.2. Technological Stack.....	13
2.3. Key Data Flows.....	14
2.3.1.1. Data Flow Diagram (DFD).....	14
2.3.2. Normalized Database Schema.....	15
3. Functional Requirements & Scope.....	16
3.1. Minimum Viable Product:.....	16
3.2. Non-Functional Requirements (NFRs).....	17
3.3. Out of Scope / Future Enhancements.....	18
4. Monitor, Evaluation, Assumptions & Dependencies.....	19
4.1. Technical Evaluation:.....	19
4.1.1. Grayscale Rollout & A/B Testing:.....	19
4.1.2. Emergency Rollback (ER) & Golden Release:.....	19
4.1.3. Priority Matrix.....	19
4.2. Monitoring.....	20
4.3. Assumptions.....	20
4.4. External Dependencies.....	21
5. Project Management & Team Contributions.....	22
5.1. Project Timeline.....	22
5.2. Team Members Role.....	22
5.3. Recommendations.....	23

1. Introduction

Duit on Track is an AI powered financial decision intelligence platform designed to help Malaysian university students make smarter spending decisions before money is spent.

The system addresses a common financial challenge among students: repeated micro spending decisions such as food delivery, transportation, entertainment and impulse purchases that gradually lead to overspending and financial instability before the end of the month.

Unlike traditional budgeting applications that only record transactions after spending happens, Duit on Track acts as a real time financial decision coach. The system evaluates spending decisions before they are made by using Z.AI's GLM as its core reasoning engine.

The strategic solution behind Duit on Track is to place an AI reasoning layer between the user and their spending decisions. This enables students to receive:

- Real-time spending recommendations
- Financial consequence simulations
- Savings goal impact analysis
- Personalized financial coaching insights

By combining structured financial data (budget, expenses, savings goals) and unstructured user inputs ("Coach AI I beli ni?"), Duit on Track provides context-aware recommendations to help users optimize their spending behavior and improve their financial outcomes.

The primary objective of Duit on Track is to reduce overspending, improve budgeting habits, and increase savings efficiency among university students through intelligent decision support.

1.1. Purpose

This System Analysis Document (SAD) outlines the technical scope, system architecture, data flow design, and implementation strategy behind the development of Duit on Track.

The purpose of this document is to provide a structured technical reference for the development team, testers, and stakeholders regarding how the system is designed, how its components interact, and how Z.AI's GLM is integrated as the core decision making engine.

The key elements of the system covered in this document are as follows:

1. Architecture

- a. Duit on Track adopts a cloud-based client-server architecture with a React frontend, Python Flask backend, Firebase database, and the Z.AI GLM reasoning layer. Hosting is handled via Vercel (frontend) and Render (backend). The modular structure ensures scalability, maintainability and clear separation of responsibilities between system components.

2. Data Flows

- a. The system manages data flow from user input through financial data retrieval, prompt construction, GLM processing, response parsing, and result delivery. All financial data, decision records, and monthly report data are stored persistently in Firebase for coaching analysis.

3. Model Process

- a. The model process demonstrates the complete workflow of Duit on Track's core decision-making features, including:
 - "Coach AI?" spending decision evaluation with Yes / No / Wait verdict
 - Mood Mode system-prompt switching (Save Mode vs. Spend Mode)
 - Behavioral pattern detection from monthly decision history
 - Monthly financial coaching report generation via batched GLM call

4. Role of Reference

- a. This document serves as the main technical reference for developers, QA testers, and project stakeholders to understand the system architecture, API interactions, data processing logic, and AI reasoning workflow of Duit on Track. It ensures that all team members share a unified understanding of the technical implementation throughout the development lifecycle.

1.2. Background

University students in Malaysia often manage their finances under limited and inconsistent income sources such as monthly parental allowances, education loans (PTPTN), and part-time earnings such as tutoring sessions. On average, students operate within a monthly budget of RM400–RM800, which must cover essential expenses including food, transportation, academic materials, and personal spending.

A major financial issue among students is not caused by large single purchases, but by the silent compounding of repeated small daily decisions. For example: a RM6 bubble tea three times a week amounts to RM72 per month; a RM15–25 Grab ride to class instead of the RapidKL bus; a RM30 impulse Shopee purchase triggered by a flash sale. These micro-decisions accumulate to overspending of RM80–120 per month on average, yet no existing tool intervenes at the moment of decision to prevent it.

1. Previous Version:

- a. The existing financial management approach used by students operates across multiple disconnected platforms:
 - Banking applications for checking balances: shows numbers, not guidance
 - E-wallet applications for making payments: records transactions, no advice
 - Manual budgeting tools such as spreadsheets or note taking apps: requires discipline students do not consistently maintain

These systems function independently and do not provide integrated financial decision support. None of them answer the question students ask themselves multiple times every day: "Coach AI I beli ni?"

2. Changes in Major Architectural Components

- **AI Reasoning Integration:** Traditional financial apps only record and display transactions. Duit on Track integrates Z.AI GLM as a reasoning engine to evaluate spending decisions in real time, before the purchase occurs.
- **Centralized Decision Engine:** A Flask backend decision engine collects user financial data, income schedule, spending history, and savings goals to construct structured prompts for GLM analysis.
- **Mood Engine:** A Mood Mode service maintains the student's current coaching personality state Save Mode (strict, conservative) or Spend Mode (permissive, empowering) and injects the appropriate system prompt into every GLM call.
- **Monthly Coach Report Service:** A dedicated service aggregates a month of decisions into a single batched GLM call, producing a warm behavioral coaching summary with pattern detection and one specific experiment for the following month.

3. New capabilities that are only introduced

- **"Coach AI?" Pre-Purchase Advice:** Students ask a spending question and receive a verdict with reasoning, RM impact, and a cheaper alternative before spending.
- **Mood-Adaptive Coaching:** The toggle between Save and Spend mode fundamentally changes GLM's decision bias and coaching tone to match the student's financial context.
- **Savings Goal Impact Analysis:** Every verdict includes the quantified RM impact on the student's active savings goal (e.g., "delays Langkawi trip by 3 weeks").
- **Monthly Behavioral Coaching Reports:** Students receive an AI-generated coaching summary identifying spending patterns and providing one specific behavioral experiment for next month.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Student	Use the system to evaluate spending decisions and monitor financial behavior	Fast, clear and useful financial guidance
Development Team	Design, build and maintain the application	Clear architecture, modular codebase, stable APIs, versioned prompt templates and well documented data contracts
QA Team	Test system reliability and AI response quality	Well-defined test cases, predictable GLM output schema, and clear fallback behaviour documentation

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web Application (React frontend, Python Flask backend)
Architecture	Cloud-based Client-Server with Z.AI GLM as the AI Service Layer

Duit on Track is structured as a client-server web application. The React frontend communicates with a Python Flask backend hosted on Render via REST API calls. The backend is organized into four logical service modules: the Decision Engine (the core "Coach AI?" processor), the Budget State Service (maintains real-time budget snapshot, days remaining, income events, goal progress), the Mood Engine (manages the student's active Save or Spend mode and injects the correct Z.AI GLM system prompt), and the Report Service (triggered at month-end, generates the Monthly Coach Report via a single batched GLM call). All persistent data is stored in Firebase Firestore.

2.1.2 LLM as Service Layer

Z.AI GLM is integrated as the dedicated AI Service Layer not a generic AI module. It is responsible for all intelligent reasoning outputs in Duit on Track:

- "Coach AI?" verdict generation: interpreting the student's spending question with full budget context to produce a Yes / No / Wait verdict with reasoning, projected RM impact, and a Malaysian-context cheaper alternative
- Mood Mode personality switching: the system prompt changes entirely based on Save or Spend mode, making GLM strict and conservative in Save Mode and permissive and empowering in Spend Mode
- Monthly Coach Report narrative: synthesising the student's full month of decision history into a warm coaching summary with behavioral pattern identification, top 3 good and harmful decisions, and one specific experiment for next month
- Spending alternative suggestions: proactively recommending cheaper Malaysian-context alternatives (e.g., "Mr DIY version at RM45 instead of RM120 hoodie")

2.1.3 Dependency Diagram

Prompt Construction (Decision Engine)

- Select system prompt from Mood Engine (Save/Spend) (~200 tokens)
- Fetch budget data from Firebase (budget left, days left, income events, savings goal)
- Include last 5 spending decisions (summarized if needed)
- Append user query (e.g., "Coach AI I beli AirPods RM450?")
- Context capped at 1,500 tokens; older history replaced with summary if exceeded

Context Window Structure

- System prompt: ~200 tokens
- Budget state: ~150 tokens
- Recent decisions: ~300 tokens
- User query: ~50 tokens

- Max input: 1,500 tokens
- Reserved output: 500 tokens

GLM API Call

- Flask backend sends POST request to Z.ai GLM API
- Timeout: 5s → fallback from cache if failed
- Rate limit: 30 requests/user/day

Response Parsing

- GLM returns JSON: { verdict, reasoning, impact_rm, alternative }
- Validate response; fallback if invalid
- Save decision to Firebase Cloud Firestore and return to frontend

Token Cost Controls

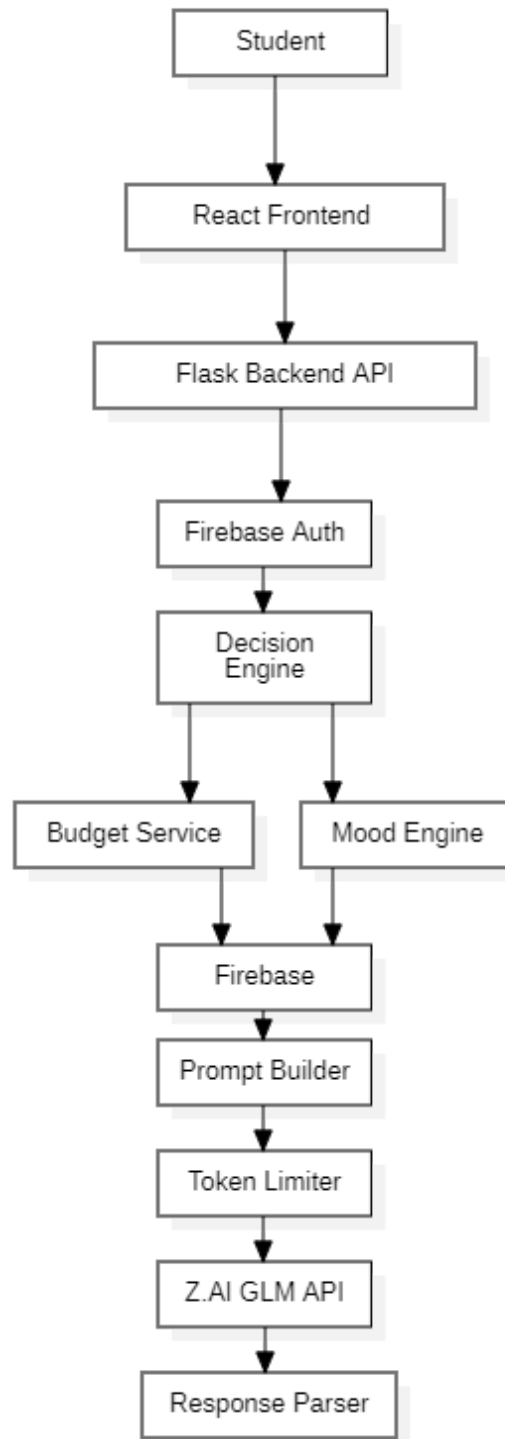
- Same query within 6 hours → serve from cache
- Monthly report uses 1 batched GLM call (~1,200 tokens)
- Prompt templates are version-controlled

Major API Calls

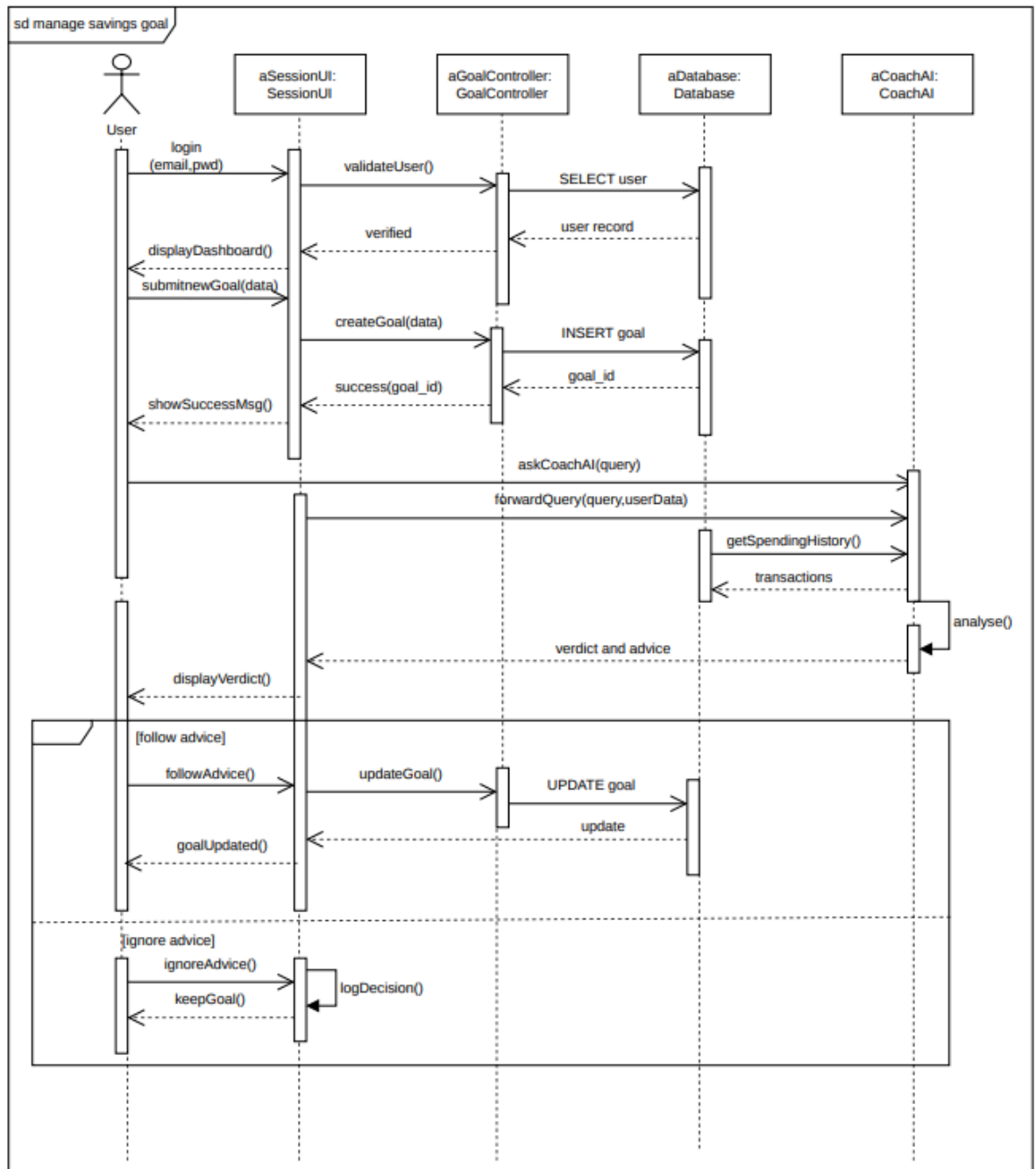
- React Frontend → Flask Backend (POST /decision/check)
- Flask → Mood Engine (get system prompt)
- Flask → Budget Service (fetch budget state)
- Flask → Cache (check/store responses)
- Flask → Z.AI GLM API (decision request)
- Flask → Firestore (save decision)
- React → Flask (PATCH /decision/action)
- Report Service → Firestore + GLM API (monthly report)

Mood Prompt Modes

- Save Mode: stricter, prioritizes saving, suggests cheaper alternatives
- Spend Mode: more flexible, allows spending with smarter choices



2.2.2. Sequence Diagram



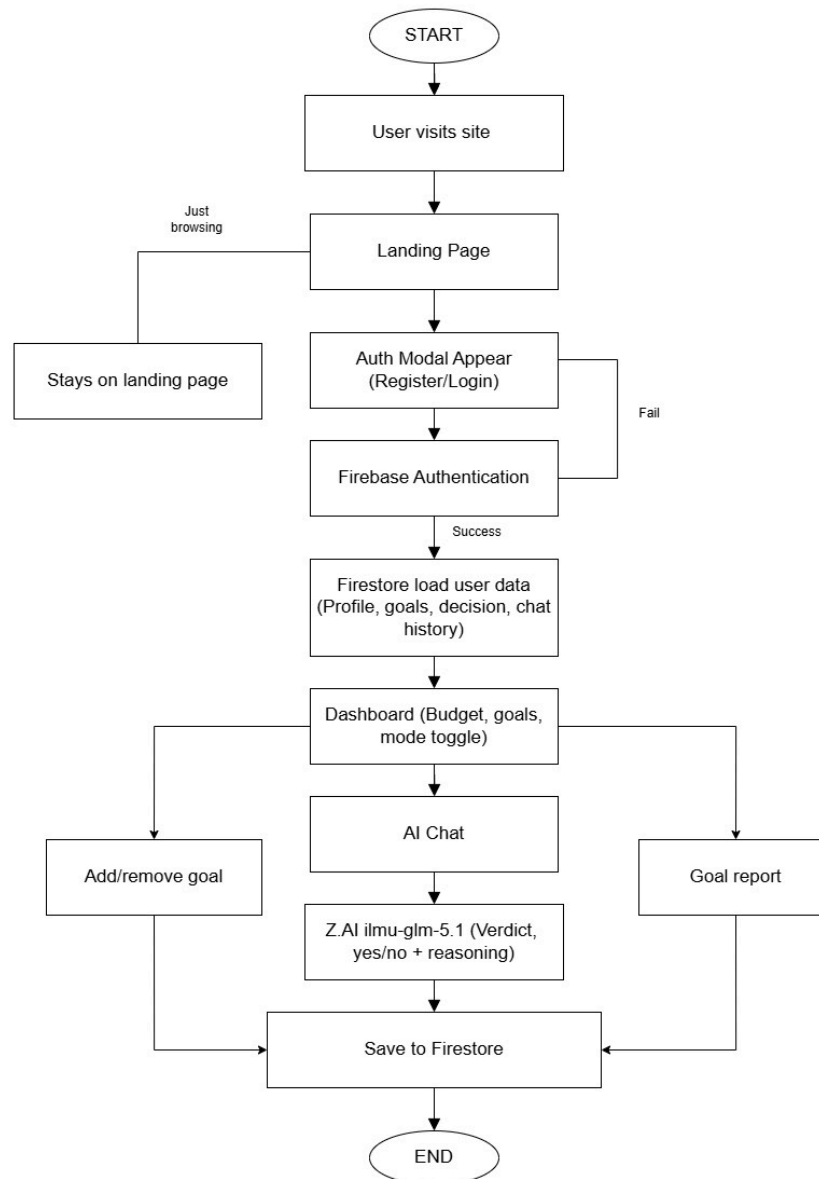
2.2. Technological Stack

Layer	Technology	Justification
Frontend	React (JavaScript)	Component-based SPA enabling fast iteration; Vercel deployment for instant preview and zero-config hosting; students access via any device browser
Backend	Python Flask	Lightweight REST API framework; native Python support simplifies Z.AI GLM integration, JSON parsing, and Firebase Admin SDK usage; hosted on Render
AI / LLM Layer	Z.AI GLM API	Mandatory per UMHackathon 2026 rules; provides the context-aware reasoning essential for "Coach AI" verdicts and Mood Mode personality switching
Database	Firebase Firestore	NoSQL document store — flexible schema suits the varied data shapes of decisions, budget state, goals, and reports; Firebase Auth provides built-in authentication; real-time listeners enable instant dashboard updates
Frontend Hosting	Vercel	Zero-config deployment for React apps; automatic CI/CD from GitHub; global CDN for fast load times for Malaysian student users
Backend Hosting	Render	Simple Flask deployment with automatic deploys from GitHub; free tier sufficient for hackathon load; supports environment variable management for GLM API keys
Auth	Firebase Auth	Built-in authentication with Google OAuth; JWT tokens validated server-side on every Flask request; low friction onboarding for students with existing Google accounts

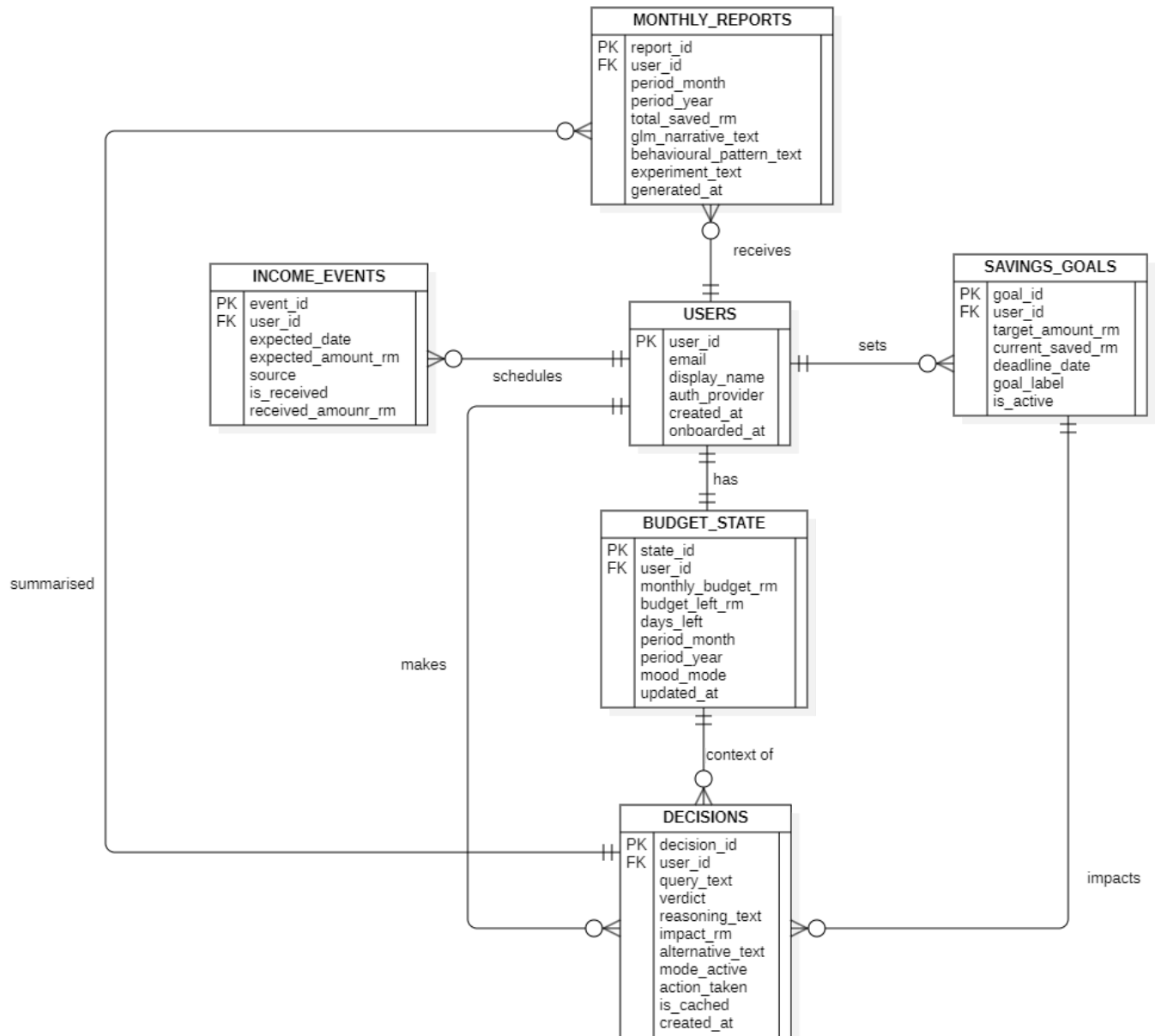
2.3. Key Data Flows

The following describes how data moves through the Duit on Track system and how it is structured and stored.

2.3.1.1. Data Flow Diagram (DFD)



2.3.2. Normalized Database Schema



3. Functional Requirements & Scope

3.1. Minimum Viable Product:

#	Feature	Description
1	"Coach AI?" Decision Check	Student type a spending question before any purchase. Z.AI GLM returns a Yes / No / Wait verdict with context-aware reasoning grounded in budget remaining, days left, upcoming income, and savings goal. Response includes projected RM impact and a Malaysian-context cheaper alternative. This is the signature feature and core identity of Duit on Track pure LLM reasoning that no rule engine can replicate.
2	Mood Mode: Save ↔ Spend Toggle	A single toggle on the student's dashboard that switches Duit on Track's entire coaching personality. Save Mode (strict, conservative) is used during tight weeks, exam crunch, or pre-payday periods. Spend Mode (permissive, empowering) is used during semester breaks or good earning months. The toggle changes the Z.AI GLM system prompt injected into every subsequent decision call.
3	Monthly Coach Report	At month-end, a one-screen coaching summary is generated via a single batched Z.AI GLM call. Contents: total RM saved from followed advice; top 3 decisions that helped; top 3 decisions that hurt; one behavioural pattern GLM detected from the month's history; one specific experiment for next month.
4	Profile Setup & Budget State Initialisation	On first launch, student inputs monthly budget, income sources (allowance amount + tutoring schedule), and savings goal (target RM + deadline). This populates Firebase Firestore forming the context the Decision Engine passes to GLM with every verdict request.
5	Impact Tracker Dashboard	Home screen displays: Budget Left (RM), Days Left until month-end, Goal Progress bar, cumulative "You've saved RMX this month" counter, Mood Mode toggle, and the quick-input "Coach AI?" box. Updated live after every action recorded.

3.2. Non-Functional Requirements (NFRs)

Quality	Requirement	Implementation
GLM Response Latency	"Coach AI" verdict must be returned within 4 seconds at p95 under normal load	5-second hard timeout on Z.AI GLM API call in Flask; streaming response shown token-by-token on React frontend; fallback served from server-side cache on timeout
Scalability	System must handle concurrent usage spikes during semester-start weeks and exam periods	Render auto-scales Flask instances on load; Vercel CDN distributes React static assets globally; Firebase Firestore scales horizontally by design
Reliability	Decision records must not be lost even if GLM call fails	Firestore write is attempted after GLM response is received; on GLM failure, a fallback verdict is returned and the failed attempt is logged — no orphaned or partial decision records
Token Latency	Z.AI GLM API calls must return a response within 3 seconds under normal load at p95	5-second timeout for async GLM calls in Flask; loading state shown on React frontend while awaiting response; streaming display progressively renders verdict text
Cost Efficiency	Average token cost per user session must not exceed a defined budget threshold	Prompts capped at 1,500 tokens input; identical queries served from server-side cache for 6 hours; monthly report generated as single batched call; max 30 GLM calls per user per day enforced via Flask rate-limit counter

Privacy & Security	Student financial data must not be transmitted to GLM API as raw personally identifiable information	Budget and goal figures sent as numeric values only; no student name, IC number, or bank details included in GLM prompts; Firebase Auth enforces authenticated access to all Firestore collections; HTTPS enforced across Vercel and Render
--------------------	--	---

3.3. Out of Scope / Future Enhancements

The following are planned for the product's grand vision but are not part of the UMHackathon 2026 deliverable:

- Peer comparison: anonymised spending benchmarks against students at the same university
- Group savings goals: shared Langkawi trip fund with friends
- Voice input via native speech-to-text for hands-free "Coach AI" queries
- Gamification layer: decision streaks, savings achievement badges

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

New Z.AI GLM prompt templates, particularly updates to the Save Mode and Spend Mode system prompts will be released to 5% of users initially via a feature flag stored in Firebase Firestore. The system monitors GLM response quality (verdict schema validity rate), user engagement rate (percentage of users who record an action after receiving a verdict), and token usage per session. If no regressions are detected over 48 hours, rollout increases: 5% → 25% → 50% → 100%.

A/B testing is used to compare prompt tone strategies for example, directive tone ("No. Do not buy it.") versus coaching tone ("Wait here is why this hurts your Langkawi goal."). Success metric: `action_taken = 'followed'` rate. The higher-performing prompt variant is promoted to 100%.

4.1.2. Emergency Rollback (ER) & Golden Release:

GitHub Actions tags each successful deployment as a candidate release. After passing all automated tests and a 1-hour canary period with no critical alerts, the release is promoted to Golden Release. If a P1 trigger condition is met, the Render and Vercel deployments are rolled back to the Golden Release within 5 minutes using GitHub Actions rollback workflow. Student data in Firebase Firestore is never rolled back only application code. Students mid-session see a brief "Duit on Track is updating your data is safe" banner.

4.1.3. Priority Matrix

4.1.3.1.

Priority Level	Trigger Condition	Action Taken
P1 — Critical	Z.AI GLM API error rate > 5% OR Firestore decision write failure rate > 1% over 5 minutes	Emergency rollback to Golden Release on Render and Vercel; activate server-side cache fallback mode

		(cached verdicts only); immediate alert to on-call team member
P2 — High	"Coach AI" response latency > 6 seconds at p95 over 10 minutes	Scale up Render Flask instances; investigate Z.AI GLM API latency; notify development team Slack channel
P3 — Medium	Impact Tracker displays incorrect saved amount for more than 5% of users	Investigate Firestore aggregation logic; recompute monthly_impact_saved from decisions collection; notify affected users
P4 — Low	Server-side cache hit rate falls below 50%	Audit cache key strategy; review TTL settings; extend cache duration for stable query patterns

4.2. Monitoring

- Vercel Analytics: React frontend page load time, core web vitals, and deployment error logs
- User Engagement Monitor: weekly report on percentage of users who record an action after receiving a verdict. Target: above 60% action recording rate. Drop below target signals friction in the Action Taken UX flow

4.3. Assumptions

- Students have a stable mobile internet connection when submitting "Coach AI" queries the app requires connectivity for GLM calls.
- Z.AI GLM API is available and performant as per UMHackathon 2026 infrastructure provisions for the development and demo period.
- Students will manually log income events (allowance receipt, tutoring session completed) automated bank sync is out of scope for this hackathon build.
- The student's monthly budget period aligns with the calendar month custom period reset is a future enhancement.
- Students consent to their anonymised decision patterns being aggregated for future coaching model improvements, disclosed during onboarding.

4.4. External Dependencies

Tools	Purpose	Risks
Z.AI GLM API (Mandatory)	Core LLM reasoning engine for all "Coach AI" decision verdicts, Mood Mode personality switching, and Monthly Coach Report narrative generation	High: GLM outage or rate-limit causes core product failure. Mitigation: server-side cached fallback verdicts for common query patterns; user shown static advice with graceful degradation message; exponential backoff retry (3 attempts: 1s / 2s / 4s intervals)
Firebase Firestore	Persistent NoSQL document store for all user data, decisions, budget state, goals, income events, and monthly reports	High: Firestore outage halts all data reads and writes. Mitigation: Firebase has 99.95% SLA; client-side Firestore SDK caches recent reads locally for brief offline resilience
Firebase Auth	User authentication, JWT issuance, Google OAuth for low-friction student onboarding	Medium: Auth outage blocks new logins. Mitigation: existing sessions continue via locally cached JWT until expiry
Vercel	Frontend hosting and deployment for the React application	Low: Vercel outage affects frontend access but not data integrity; 99.99% uptime SLA
Render	Backend hosting for the Python Flask REST API and all GLM integration logic	Medium: Render outage halts all API calls. Mitigation: Render auto-restart policy; rollback to Golden Release via GitHub Actions within 5 minutes

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline

Period	Task
Day 1 - 2	Planning & Design
Day 3 - 5	Backend & Z.AI GLM Integration
Day 6	Frontend Development
Day 7 - 8	Testing & Simulation
Day 9	Report Service & Integration QA
Day 10	Polish, Demo & Documentation

5.2. Team Members Role

Member	Role
MARIAM SHAZLINDA BINTI MOHAMED RAIHAN	Backend & API
DANIA BATRISYIA BINTI SUHAIMI	Frontend UI & Testing
MUHAMMAD IBRAHIM SHAH BIN ABDUL RAZAK	Testing & Documentation

5.3. Recommendations

- Firebase Real-Time Listeners: Use Firestore onSnapshot listeners in the React Dashboard to update the Impact Tracker and Budget Left display in real time whenever a decision document is written no polling required.
- Prompt Template Versioning: Store all Save Mode and Spend Mode system prompts as Firestore documents with a version field. This enables prompt A/B testing and rollback without requiring code deployments to Render.
- Income Event Reminder: A scheduled Flask job at 8am each day checks income_events for events expected on that day and sends a browser push notification: "Did you receive your tutoring payment today? Update your budget so Coach can advise you better." This keeps the budget state accurate without requiring students to remember to update manually.